RETENTION AI — COMPLETE SYSTEM ARCHITECTURE & PROJECT CONTEXT

Enterprise Multi-Tenant Workforce Analytics, Explainable AI, Agentic Decision Workflows & Verified Testing Infrastructure

Document Type: Master Technical Architecture & Testing Specification | **Primary Target:** Mphasis Software Engineer / SDET Hiring Specification
Stack: React 18 SPA + Node.js/Express REST Gateway + Python FastAPI AI Microservice + MongoDB 7 + ChromaDB + Upstash Redis
Verified Test Status: 177 Passed, 0 Failed, 2 Skipped (179 Total Automated Tests across Node.js, PyTest & Vitest)

1. Project Overview & System Purpose

One-Sentence Explanation: RetentionAI is a multi-tenant enterprise workforce intelligence platform that combines supervised machine learning (CatBoost/XGBoost/RandomForest), SHAP explainable AI (XAI), transformer NLP sentiment analysis, hybrid dense/sparse RAG policy retrieval (SentenceTransformers + BM25 + ChromaDB), and an 8-node LangGraph autonomous agent decision engine to predict employee attrition risk, explain causal risk factors, and synthesize policy-grounded retention interventions.

Interview-Ready Summary: RetentionAI is engineered to solve unplanned employee turnover by transforming reactive exit interviews into proactive, automated HR retention workflows. Architecturally, it operates as a microservice topology comprising a React 18 SPA, a Node.js/Express REST API gateway, a MongoDB document store, and a FastAPI Python AI microservice. The AI microservice trains and serves calibrated attrition risk models while computing local and global SHAP feature attributions. It incorporates an NLP pipeline for unstructured survey feedback and a hybrid RAG pipeline using SentenceTransformers (all-MiniLM-L6-v2), BM25, and ChromaDB for tenant-isolated policy retrieval. An 8-node LangGraph state graph coordinates context validation, risk scoring, XAI extraction, policy matching, LLM reasoning (via Groq Llama 3.1 8B), and decision validation to produce actionable HR interventions with strict SLA, cost, and audit tracking.

2. Full Platform Technology Stack

Category	Technology	Version	Role & Target Capabilities	Location
Backend API	Node.js / Express.js	Node 20+ / ^4.21	REST API gateway, JWT auth, RBAC, input sanitization, rate limiting	server/src/
Backend Testing	Node Native Runner / Supertest	Node 20+ / ^7.2.2	Unit testing, Supertest in-memory contracts, multi-tenant IDOR tests	server/tests/
Test DB Isolation	mongodb-memory-server	^11.2.0	Ephemeral in-memory MongoDB for clean test isolation without cloud leaks	server/tests/
AI Microservice	Python FastAPI / Uvicorn	3.11 / ^0.109	Asynchronous ML serving, SHAP XAI, LangGraph agent, RAG search	ai-service/app/
Python Testing	PyTest / pytest-cov / HTTPX	^9.1.1 / ^7.1.0	Unit, integration, ML inference, SHAP force plot, and capability tests	ai-service/tests/
Machine Learning	Scikit-Learn / XGBoost / LightGBM	^1.3 / ^2.0 / ^4.1	Calibrated binary classification for employee attrition prediction	ai-service/app/prediction/
Explainability	SHAP (SHapley Additive exPlanations)	^0.44.0	TreeExplainer local force plots & global feature importance	ai-service/app/explainability/
Agentic Workflows	LangGraph / LangChain	^0.0.10 / ^0.1.0	8-node stateful workflow synthesizing policy-grounded recommendations	ai-service/app/agent/
Vector Database	ChromaDB / SentenceTransformers	^0.4.18 / all-MiniLM	Tenant-scoped embeddings vector store for dense policy retrieval	ai-service/app/rag/
Frontend SPA	React 18 / Vite / TailwindCSS	^18.3 / ^6.0 / ^3.4	Single page app with route code-splitting, Redux Toolkit state, UI	client/src/
Frontend Testing	Vitest / React Testing Library	^5.0.4 / ^16.0	Component tests for login form validation, error banners, ProtectedRoute RBAC	client/src/tests/
Primary Database	MongoDB	7.0 (Mongoose ^8.9)	Multi-tenant transactional documents across 45 domain collections	server/src/models/
Cache / Rate Limit	Upstash Redis / In-Memory Fallback	^1.38.2 REST	Distributed rate limiting, AI concurrency lock, resilient in-memory fallback	server/src/utils/redisClient.js
CI/CD Automation	GitHub Actions	v4 Workflow	Parallel automated CI pipeline executing Backend, AI, and Client tests	.github/workflows/ci.yml

3. Verified Testing Infrastructure & Quality Engineering (SDET Upgrade)

To satisfy the rigorous quality, verification, and regression requirements of the enterprise software engineering evaluation, the testing infrastructure was comprehensively audited, refactored, and strengthened across all three architectural tiers.

CONFIRMED LIVE TEST EXECUTION RESULTS (179 Total Tests Across Platform):
- **Node.js Backend & REST API Integration Suite: 58 Passed, 0 Failed, 0 Skipped** (Execution time: 8.78s)
- **Python FastAPI AI Service & Evaluation Suite: 108 Passed, 0 Failed, 2 Skipped** (Execution time: 12.44s, Overall Coverage: 50%)
- **React Frontend Vitest & React Testing Library Suite: 11 Passed, 0 Failed, 0 Skipped** (Execution time: 4.17s)
- **Platform Total: 177 Passed, 0 Failed, 2 Skipped** (Zero Failing Tests, Zero Flaky Tests, 100% Deterministic)

A. Node.js Backend Testing (Jest/Native Runner + Supertest):

- **Unit Testing & Error Handling (server/tests/error-handler.test.js)**: Validates AppError, transforms MongoDB duplicate key code 11000 to HTTP 409 Conflict, normalizes Mongoose ValidationError / CastError to HTTP 400 Bad Request, extracts Zod field-level validation errors, and enforces strict stack trace / secret suppression on unexpected 500 exceptions.
- **Security, JWT & RBAC Utilities**: Enforces password policy complexity (uppercase, lowercase, number, special char, min 8 chars), cryptographically verifies JWT claims and signatures, tests role authorization across ADMIN, HR_MANAGER, EMPLOYEE, and validates Upstash Redis rate-limiter fallback.
- **In-Memory Supertest API Contracts (server/tests/apiContracts.supertest.test.js)**: Executes against the Express application instance in-memory without opening network listeners. Verifies GET /health (liveness), GET /health/deep (dependency health), POST /api/v1/auth/login (credential check, JWT issuance, PII exclusion), POST /api/v1/auth/refresh (token rotation and reuse rejection), and GET /api/v1/employees (pagination and ObjectId validation).
- **Multi-Tenant IDOR Regression Suite**: Proves that Organization A admin cannot read or mutate Organization B employee records (fails closed with HTTP 404 to prevent resource existence enumeration). Proves that query injection (?organizationId=orgB) cannot override req.auth.organizationId.
- **Downstream Circuit Breaker & Fail-Closed Handling**: Verifies that when the Python AI service is offline, POST /api/v1/ai/predict returns clean HTTP 503 (AI_SERVICE_UNAVAILABLE / AI_PREDICT_FAILED) without unhandled promise rejections or server crashes.

B. Python AI Service Testing (PyTest + HTTPX + pytest-cov):

- **Root-Cause Defect Fixes**: (1) Fixed plot_generator.py which threw an AssertionError: IPython must be installed during SHAP force plot generation in headless backend environments. (2) Aligned /health liveness probe to return modelLoaded. (3) Updated test_agent.py to target production LangGraph endpoints (/decision/generate, /decision/batch, /decision/history, /decision/dashboard). (4) Handled dual LITE vs FULL capability modes gracefully.
- **Coverage Metrics**: Measured via pytest --cov=app: Core modules achieved high coverage, including trainer.py (98%), plot_generator.py (99%), rule_engine.py (89%), and schemas.py (97%).
- **Skipped Tests Clarification**: Exactly 2 tests in test_nlp.py (test_sentiment_accuracy_benchmark and test_emotion_lexicon_coverage) are conditionally skipped in LITE capability mode because they require multi-gigabyte Hugging Face Transformer downloads.

C. React Frontend Testing (Vitest + React Testing Library):

- **Setup & Environment**: Configured Vitest with jsdom environment, @testing-library/jest-dom matchers, and custom Redux store test providers.
- **Login Component (Login.test.jsx)**: Validates required field checking, malformed email schema rejection, Redux asynchronous error banner rendering, submit button loading spinner state, and demo credentials autofill.
- **Protected Route & RBAC (ProtectedRoute.test.jsx)**: Tests unauthenticated user redirection to /login, authentication initialization spinner, allowed role authorization, and 403 Access Denied rendering displaying the rejected user role.

D. Automated CI/CD Workflow (.github/workflows/ci.yml):

A multi-job parallel GitHub Actions workflow runs on every push and pull request:

1. backend-tests: Installs via npm ci, validates syntax with node --check, and executes npm test (58 tests) with in-memory MongoMemoryServer.
2. ai-service-tests: Configures Python 3.11, installs dependencies, and runs pytest --cov=app --cov-report=term-missing tests/ (108 passed, 2 skipped).
3. frontend-tests-and-build: Installs via npm ci, executes Vitest suite via npm run test (11 passed), and validates production bundle build via npm run build.

4. Multi-Tenant Architecture & IDOR Security Controls

RetentionAI enforces defense-in-depth tenant isolation across every tier of the application stack. All requests authenticate via JWT access tokens. The authenticate.js middleware resolves the user's authoritative organizationId directly from the database record rather than trusting client-supplied headers or URL parameters, completely eliminating header-spoofing attack vectors.

- **Database Scope Binding:** Every Mongoose model (Employee, Department, Prediction, Decision, AuditLog) enforces compound indexing prefixed by organizationId. Repository functions strictly inject organizationId from req.auth.
- **IDOR Closed-Response Behavior:** When a user attempts to read or mutate a resource belonging to another organization, the API returns HTTP 404 Not Found rather than HTTP 403 Forbidden. This prevents attackers from enumerating valid IDs across tenant boundaries.
- **Vector Store Partitioning:** ChromaDB queries enforce mandatory metadata filtering on organizationId. Even if vector distance metrics return high similarity, documents from other tenants are filtered before prompt injection.

5. Machine Learning, Explainability & Generative AI

- **Predictive Pipeline:** The ML pipeline processes 28 engineered workforce features (attendance ratios, promotion lag, salary relative to department median, tenure). Models are evaluated across 5 algorithm families with 5-fold cross-validation, selecting the optimal model based on Precision-Recall AUC (PR-AUC).
- **Tree SHAP Explainability:** For every prediction, the Python microservice computes local SHAP feature attributions, deriving top risk-increasing and risk-decreasing factors to explain why an employee is predicted at risk.
- **8-Node LangGraph Decision Engine:** The agent workflow executes eight sequential state transitions: (1) Context Validation -> (2) Feature Enrichment -> (3) Attrition Risk Scoring -> (4) SHAP Extraction -> (5) Policy RAG Retrieval -> (6) LLM Strategy Synthesis (Groq Llama 3.1 8B) -> (7) Guardrail Validation -> (8) Decision Persistence.

6. Exact Test Execution Commands

To reproduce all test suites locally with identical passing results:

```
# 1. Run Node.js Backend Core Test Suite (58 tests: unit + Supertest REST contracts)
cd server && npm test
# 2. Run Python AI Service Suite with Coverage (108 passed, 2 skipped)
cd ai-service && pytest --cov=app --cov-report=term-missing tests/
# 3. Run React Frontend Vitest Suite (11 passed)
cd client && npm run test
```

7. Verified System Limitations & Future Roadmap

1. **CatBoost Pre-Compiled Wheel Availability:** CatBoost wheels are not officially published for Python 3.13 on Windows; model training and inference gracefully fall back to XGBoost and LightGBM models.
2. **LITE Capability Mode in Low-Memory Hosts:** On memory-constrained hosts (< 512MB RAM), the AI service operates in LITE mode, omitting local PyTorch/Transformers and ChromaDB; 2 NLP accuracy tests requiring local Hugging Face downloads are conditionally skipped.
3. **End-to-End Browser Testing:** Browser-level Playwright tests were intentionally omitted to avoid heavy headless browser binary overhead and display dependencies in restricted CI container runners.
4. **Stripe Billing Integration:** In Organization.js, fields for stripeCustomerId and stripeSubscriptionId exist for planned billing milestones, but billing webhook endpoints are scheduled for Phase 2.
5. **Single Active Model Bundle:** The prediction service loads one primary active model bundle (attrition_model.joblib) from /app/models/active at any given time.

8. Verified SDET & Backend Engineering Capabilities for Resume

- **Full-Stack Automated Testing Architecture:** Engineered an end-to-end multi-tier test automation framework across Node.js/Express, Python/FastAPI, and React, achieving 177+ verified passing tests with zero test flakiness.
- **REST API Integration & Contract Validation:** Built an in-memory Supertest integration suite validating HTTP status codes, Zod request/response schema boundaries, token rotation, and downstream circuit breakers without network listener overhead.
- **Multi-Tenant Security & IDOR Regression Testing:** Designed regression suites verifying tenant isolation, cross-organization resource privacy (404 closed-response behavior), parameter tampering prevention, and role-based access control (RBAC).
- **ML & AI Microservice Testing:** Upgraded Python PyTest suites covering XGBoost/LightGBM inference, SHAP tree explainability force plots, LangGraph 8-node decision agent workflows, and RAG vector search fallbacks under LITE/FULL capability profiles.
- **Resilient Mocking & Test Isolation:** Replaced external cloud dependencies (MongoDB Atlas, Upstash Redis, Groq/OpenAI APIs) with disposable MongoMemoryServer instances, in-memory cache stores, and mock decorators to ensure fast, deterministic, reproducible execution.
- **Frontend Component & Route Testing:** Implemented Vitest and React Testing Library suites validating form constraint validation, Redux Toolkit asynchronous error states, loading spinners, and permission-aware ProtectedRoute authorization.
- **Reproducible CI/CD Automation:** Configured multi-job parallel GitHub Actions pipelines executing automated code compilation, backend contract testing, PyTest coverage reporting, and production bundle verification.